SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 6: Task C.3

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – FEB-2026

Contents

1. Overview of what I implemented 3
2. Candlestick chart function (with n-day aggregation option) 3
   2.1 Purpose 3
   2.2 Key requirement: n trading days per candlestick (n ≥ 1) 3
   2.3 Evidence produced 3
3. Moving-window boxplot function (rolling distribution) 5
   3.1 Purpose 5
   3.2 Implementation details 5
   3.3 Evidence produced 5
4. Less straightforward code lines 6
5. Main challenges faced 6

1. Overview of what I implemented

After completing Task C.2, I upgraded the project to version v0.2 by adding additional visualization functions to help interpret stock market data. Data visualization supports better understanding of trends and volatility, and it provides clearer insight into model inputs and market behavior. In this task, I implemented:

1. A candlestick chart function using the mplfinance library.
2. A moving-window boxplot function using matplotlib to visualize the distribution of prices across rolling windows.

All code was written with detailed comments explaining key parameters and non-trivial steps.

2. Candlestick chart function (with n-day aggregation option)

2.1 Purpose

Candlestick charts display OHLC information (Open, High, Low, Close) and are widely used to represent daily price movement. They show:

- the daily/period open and close (candle body),
- the intraperiod high and low (wicks),
- and optionally trading volume (volume bars).

2.2 Key requirement: n trading days per candlestick (n ≥ 1)

To satisfy the requirement that each candlestick can represent n trading days, I implemented an aggregation step before plotting.

Aggregation rules used (OHLCV):

- Open = first Open in the n-day group
- High = maximum High in the group
- Low = minimum Low in the group
- Close = last Close in the group
- Volume = sum of Volume in the group

This approach is consistent with standard financial candle aggregation.

2.3 Evidence produced

I generated two candlestick charts to demonstrate the option:

- Candlestick (n=1 trading day per candle) — daily candles

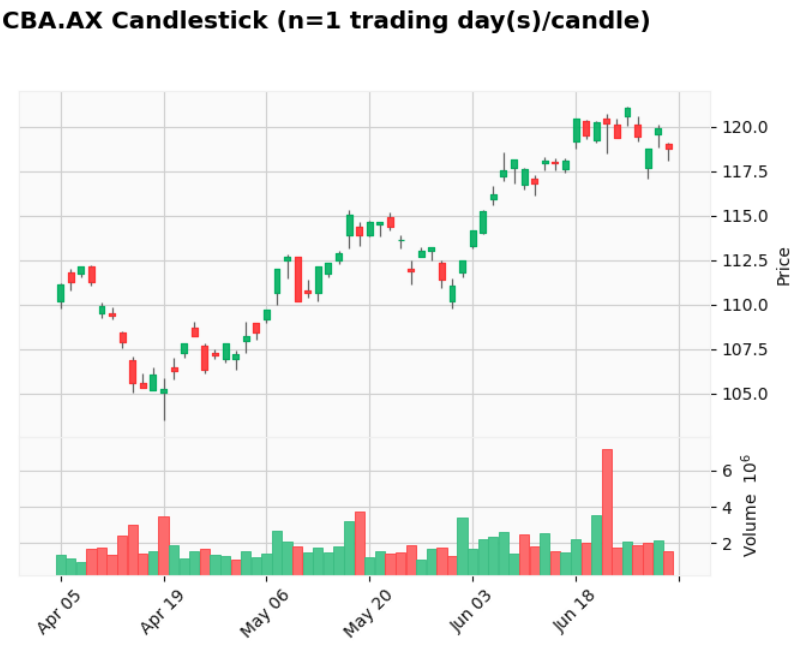

Figure C3-1: "CBA.AX Candlestick (n=1 trading day(s)/candle)"

- Candlestick (n=5 trading days per candle) — each candle summarizes 5 trading days

Figure C3-2: "CBA.AX Candlestick (n=5 trading day(s)/candle)"

3. Moving-window boxplot function (rolling distribution)

3.1 Purpose

The moving-window boxplot shows how the distribution of closing prices changes over time. This is useful for identifying changes in volatility and price range.

3.2 Implementation details

The function takes:

- window_size = number of consecutive trading days in each window (e.g., 20 days)

```
plot_moving_window_boxplot(df_raw, price_column="Close", window_size=20, step=5, company=COMPANY,
                           save_path="moving_window_boxplot.png")
```

- step = how far the window moves forward each time (e.g., 5 days)

```
plot_moving_window_boxplot(df_raw, price_column="Close", window_size=20, step=5, company=COMPANY,
                           save_path="moving_window_boxplot.png")
```

For each window:

- a boxplot is created from those window_size closing prices.
- repeating this across the series creates a “rolling distribution view”.

Why step matters:

Smaller step = more overlap (more detail but more boxes);

Larger step = less overlap (cleaner but less granular).

3.3 Evidence produced

I generated a boxplot using window=20 and step=5:

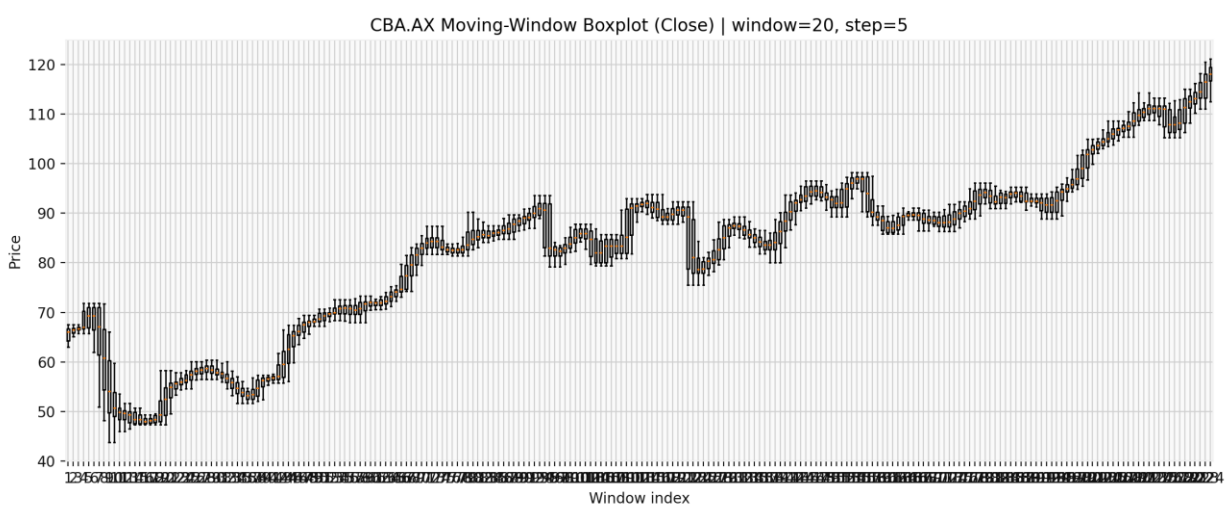

Figure C3-3: “CBA.AX Moving-Window Boxplot (Close) | window=20, step=5”

4. Less straightforward code lines

This task required research mainly in candlestick plotting and correct time-series aggregation:

1. Using mplfinance
- I learned how mpf.plot() expects the dataframe format (DatetimeIndex + OHLC columns) and how to enable volume and style options.
2. Aggregating n-day candles correctly
- The non-trivial part was implementing the OHLCV aggregation rules so the generated n-day candle remains financially valid.
3. Rolling-window boxplot
- The main implementation challenge was building a list of windows and feeding it to plt.boxplot() while controlling outliers and layout.

Reference used (online resource):

CoderzColumn candlestick tutorial (mplfinance/plotly/bokeh): https://coderzcolumn.com/tutorials/data-science/candlestick-chart-inpython-mplfinance-plotly-bokeh

5. Main challenges faced

1. Correct OHLCV formatting for plotting:
   Some yfinance outputs can have unusual column structures; I ensured the chart function uses standard OHLCV columns (Open, High, Low, Close, Volume) before plotting.
2. Making the boxplot readable:
   Rolling boxplots can become dense. I selected window=20 and step=5 as a reasonable balance between detail and readability.